Q0 Before you start; before you submit

I read Handout 5 on Turing reductions before starting, and the reductions below follow it: name the oracle, give pseudocode for the machine that uses it, show that the machine always halts, and prove correctness in both directions. I did not use Rice's Theorem anywhere. I did not collaborate with anyone on this homework.

Q1 Self assessment

The part of HW6 with the most room for improvement is Question 3(c), about replacing the oracle H by the machine M_{SH} that decides the finite language $L_{\text{SmallHalt}}$. Here is what I submitted:

> Replacing H by M_{SH} gives an ordinary Turing machine M'_G (no oracle) that makes one call $M_{SH}(\langle M_{\text{SEARCH}}\rangle, \varepsilon)$ and returns the opposite answer. Whether this "works" depends on the length of that query.
>
> **If** $(\langle M_{\textbf{Search}}\rangle, \varepsilon)$ **has** 1000 **or more characters**, it is not in $L_{\text{SmallHalt}}$ by definition, whatever M_{SEARCH} does. So M_{SH} rejects it and M'_G always accepts, regardless of whether the conjecture is true. In this case the solution does not work: M'_G's answer is not based on the behavior of M_{SEARCH} at all. (The correctness chain from part (b) breaks at the step "$(\langle M_{\text{SEARCH}}\rangle, \varepsilon) \in L_{\text{HALT}} \implies$ oracle accepts," which is no longer justified.)
>
> **If** $(\langle M_{\textbf{Search}}\rangle, \varepsilon)$ **has fewer than** 1000 **characters**, then M_{SH} agrees with H on this particular query, so the proof from part (b) goes through word for word, and M'_G is a Turing machine with no oracle that accepts on every input if the conjecture is true and rejects on every input otherwise.
>
> This sounds exciting, but it does not help us learn the answer. The reason is that the proof that $L_{\text{SmallHalt}}$ is decidable is non-constructive. $L_{\text{SmallHalt}}$ is finite, so it is decided by a machine that hard-codes the finite list of its members (as in HW5, Q6(a)), but we are not told which strings are on that list. Writing down M_{SH} would require knowing, for each of the finitely many pairs of length < 1000, whether M halts on x, and for the pair $(\langle M_{\text{SEARCH}}\rangle, \varepsilon)$ that is exactly the question of whether the Goldbach Conjecture is true. So M_{SH} exists, but nobody can actually write it down and run it without already having resolved the conjecture.
>
> In fact, the existence of a Turing machine that answers the Goldbach question was never in doubt: one of the two trivial machines "always accept" and "always reject" is correct, we just do not know which. Part (b) is interesting only because the oracle H is a single machine that would answer this and countless other open questions at once, and part (c) shows that the finite restriction $L_{\text{SmallHalt}}$ does not give us anything like that.

What was deficient: I never actually answered the question. The problem asks whether the solution from part (b) still works, and the posted solution answers in two sentences: the only query is $(\langle M_{\text{SEARCH}}\rangle, \varepsilon)$, it has fewer than 1000 characters, so M_{SH} returns the same answer as H on it and the whole correctness analysis from part (b) goes through. I instead split into two cases by the length of the query and treated both as equally plausible, so the real answer is buried in the middle of a case analysis and the rubric item "state that the query has fewer than 1000 characters" is not clearly there. I also never said why the query is short. Then I spent the second half on the remark about why this does not settle the conjecture, which is interesting but is not what was asked.

How to fix it: start with the answer. Yes, it still works. $\langle M_{\text{SEARCH}}\rangle$ is the encoding of the short program from part (a) together with a short isPrime subroutine, so even with a generous encoding it is a few hundred characters, and ε adds nothing, so the pair is well under 1000 characters. Hence $(\langle M_{\text{SEARCH}}\rangle, \varepsilon) \in L_{\text{SmallHalt}}$ if and only if $M_{\text{SEARCH}}(\varepsilon)$ halts, which means M_{SH} and H agree on the only query M_G makes. The four-line implication chains from part (b) are then unchanged with M_{SH} in place of H, and M'_G is an ordinary Turing machine that accepts if and only if the Goldbach Conjecture is true. The remark that we do not know what M_{SH} looks like can stay, but as a two-sentence note at the end, not as half of the answer.

Q2.1 Warm-up exercises (a)

No. $L_1 \cap L_2$ is not necessarily decidable.

Counterexample: $L_1 = L_{\text{HALT}}$ and $L_2 = \Sigma^*$.

$L_1 = L_{\text{HALT}}$ **is recognizable.** This was shown in lecture. The recognizer is the universal machine: on input $(\langle M \rangle, x)$, run M on x, and accept as soon as M halts. If $M(x)$ halts, the simulation finishes and the machine accepts; if $M(x)$ loops, the simulation never finishes, so the machine does not accept. So it accepts exactly the strings in L_{HALT}.

$L_2 = \Sigma^*$ **is decidable.** The machine that accepts every input immediately halts on every input and accepts exactly the strings in Σ^*.

$L_1 \cap L_2$ **is undecidable.** Since $L_{\text{HALT}} \subseteq \Sigma^*$, we have $L_1 \cap L_2 = L_{\text{HALT}} \cap \Sigma^* = L_{\text{HALT}}$, and L_{HALT} is undecidable (proved in lecture).

So L_1 is recognizable, L_2 is decidable, and $L_1 \cap L_2$ is undecidable. This disproves the claim.

The reason this can happen: intersecting with a decidable language can only remove strings, and intersecting with Σ^* removes nothing, so the undecidable part of L_1 is still there. (Part (b) shows that recognizability, on the other hand, does survive.)

Q2.2 Warm-up exercises (b)

Yes. $L_1 \cap L_2$ is always recognizable.

Since L_1 is recognizable, there is a Turing machine R_1 that accepts exactly the strings in L_1 (it may reject or loop on the others). Since L_2 is decidable, there is a Turing machine D_2 that halts on every input and accepts exactly the strings in L_2. Consider the following machine.

Input: A string x
Output: accept if $x \in L_1 \cap L_2$; otherwise reject or loop

```
1: function R(x)
2:     if D_2(x) rejects then return reject
3:     end if
4:     Run R_1(x)                    ▷ this may loop, which is allowed for a recognizer
5:     if R_1(x) accepts then return accept
6:     else return reject
7:     end if
8: end function
```

A recognizer does not have to halt on every input, so the only thing to prove is that R accepts x if and only if $x \in L_1 \cap L_2$.

If $x \in L_1 \cap L_2$, then R accepts x. Since $x \in L_2$ and D_2 decides L_2, the first call $D_2(x)$ halts and accepts, so the first **if** does not fire. Since $x \in L_1$ and R_1 recognizes L_1, $R_1(x)$ halts and accepts. So R reaches the last **if** and accepts.

If $x \notin L_1 \cap L_2$, then R does not accept x. There are two cases.

- $x \notin L_2$. Then $D_2(x)$ halts and rejects, so R rejects on the first **if**.
- $x \in L_2$ but $x \notin L_1$. Then $D_2(x)$ accepts and R goes on to run $R_1(x)$. Since $x \notin L_1$, $R_1(x)$ does not accept: it either rejects, in which case R rejects, or it loops, in which case R loops. Either way R does not accept.

So R accepts exactly the strings in $L_1 \cap L_2$, i.e., R recognizes $L_1 \cap L_2$.

(Calling D_2 first is not essential, since a recognizer may loop anyway, but it is the natural order: the decidable test is the one that is guaranteed to finish.)

Q2.3 Warm-up exercises (c)

The key observation is that *every* Turing machine recognizes some language. By definition, M recognizes L if M accepts exactly the strings in L. Given any Turing machine M, let $L(M) = \{x : M(x) \text{ accepts}\}$. This is a perfectly good language, and M accepts exactly the strings in it, so M recognizes $L(M)$. It does not matter whether M loops on some inputs, since a recognizer is allowed to loop on strings outside its language. So

$$L_{\text{REC}} = \{\langle M\rangle : M \text{ is a Turing machine}\},$$

the set of all strings that are valid encodings of Turing machines.

As in lecture, a valid encoding $\langle M\rangle$ is a string that lists the components of M: the finite set of states, the input and tape alphabets, the transition function as a finite table, and the start, accept, and reject states. Checking whether a given string has this form is a finite syntactic check, with no need to run M.

Input: A string w
Output: accept if $w \in L_{\text{REC}}$, reject otherwise

1: **function** $D_{\text{REC}}(w)$
2: Try to parse w as an encoding $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$ of a Turing machine
3: Check that Q, Σ, Γ are finite lists, that δ assigns to each pair in $Q \times \Gamma$ exactly one triple in $Q \times \Gamma \times \{L, R\}$, and that $q_0, q_{\text{acc}}, q_{\text{rej}} \in Q$
4: **if** every check passes **then return** accept
5: **else return** reject
6: **end if**
7: **end function**

D_{Rec} always halts. Its input w is a finite string, and every step is a finite check on that string: parsing it, counting entries, and comparing the entries of the transition table against the finite lists of states and symbols. D_{REC} never simulates the machine described by w.

Correctness.

- If $w \in L_{\text{REC}}$, then $w = \langle M\rangle$ for some Turing machine M (which recognizes $L(M)$), so w is a valid encoding, every check passes, and D_{REC} accepts.
- If $w \notin L_{\text{REC}}$, then w is not the encoding of any Turing machine: if it were, that machine would recognize its own language $L(M)$ and w would be in L_{REC}. So some check fails and D_{REC} rejects.

Hence D_{REC} decides L_{REC}, and L_{REC} is decidable.

(Under the alternative convention where every string is treated as the encoding of some Turing machine, say a fixed default machine for malformed strings, L_{REC} is just Σ^* and the decider is "always accept.")

Q3.1 Deciding Undecidability (a)

$L_{\varepsilon\text{-ACC}} = \{\langle M\rangle : M \text{ accepts } \varepsilon\}$ is **undecidable**. I show $L_{\text{ACC}} \leq_T L_{\varepsilon\text{-ACC}}$; since L_{ACC} is undecidable (lecture), so is $L_{\varepsilon\text{-ACC}}$.

Let O be an oracle that decides $L_{\varepsilon\text{-ACC}}$: O halts on every input, and $O(\langle N\rangle)$ accepts if and only if $N(\varepsilon)$ accepts. The idea is to turn the question "does M accept x?" into a question about a machine M' that behaves on every input, in particular on ε, the way M behaves on x.

Input: $(\langle M\rangle, x)$, where M is a Turing machine and x a string
Output: accept if $M(x)$ accepts, reject otherwise

1: **function** $M_{\text{ACC}}(\langle M\rangle, x)$
2: Construct the code $\langle M'\rangle$ of the following machine M':
function $M'(w)$: ▷ w is ignored
Run $M(x)$ ▷ $\langle M\rangle$ and x are hard-coded into M'
if $M(x)$ accepts **then return** accept **else return** reject
3: **return** $O(\langle M'\rangle)$
4: **end function**

M_{ACC} always halts. Constructing $\langle M'\rangle$ is string manipulation: M_{ACC} writes a fixed template of code with the strings $\langle M\rangle$ and x pasted in. It does not run M or M'. Then it makes one call to O, which halts because O decides $L_{\varepsilon\text{-ACC}}$, and returns that answer.

Key property of M'. For every input w, $M'(w)$ ignores w and runs $M(x)$. If $M(x)$ accepts, M' accepts. If $M(x)$ rejects, M' rejects. If $M(x)$ loops, M' never gets past the simulation, so M' loops. In particular, for $w = \varepsilon$:

$$M'(\varepsilon) \text{ accepts} \iff M(x) \text{ accepts.}$$

Correctness. By the definition of L_{ACC}, the key property, the definition of $L_{\varepsilon\text{-ACC}}$, the fact that O decides $L_{\varepsilon\text{-ACC}}$, and the code of M_{ACC},

$$\begin{aligned}
(\langle M\rangle, x) \in L_{\text{ACC}} &\implies M(x) \text{ accepts} \\
&\implies M'(\varepsilon) \text{ accepts} \\
&\implies \langle M'\rangle \in L_{\varepsilon\text{-ACC}} \\
&\implies O(\langle M'\rangle) \text{ accepts} \\
&\implies M_{\text{ACC}} \text{ accepts,}
\end{aligned}$$

$$\begin{aligned}
(\langle M\rangle, x) \notin L_{\text{ACC}} &\implies M(x) \text{ rejects or loops} \\
&\implies M'(\varepsilon) \text{ rejects or loops, so does not accept} \\
&\implies \langle M'\rangle \notin L_{\varepsilon\text{-ACC}} \\
&\implies O(\langle M'\rangle) \text{ rejects} \\
&\implies M_{\text{ACC}} \text{ rejects.}
\end{aligned}$$

So M_{ACC} decides L_{ACC} using an oracle for $L_{\varepsilon\text{-ACC}}$, that is, $L_{\text{ACC}} \leq_T L_{\varepsilon\text{-ACC}}$. If $L_{\varepsilon\text{-ACC}}$ were decidable, we could replace O by its decider and get a decider for L_{ACC}, which does not exist. Therefore $L_{\varepsilon\text{-ACC}}$ is undecidable.

Q3.2 Deciding Undecidability (b)

$L = \{(\langle M_1\rangle, \langle M_2\rangle) : \varepsilon \in L(M_1) \cup L(M_2)\}$ is **undecidable**. Here $L(M_i)$ is the set of strings M_i accepts, so $(\langle M_1\rangle, \langle M_2\rangle) \in L$ exactly when $M_1(\varepsilon)$ accepts or $M_2(\varepsilon)$ accepts (or both). I show $L_{\text{ACC}} \leq_T L$; since L_{ACC} is undecidable, so is L.

Let O be an oracle that decides L: O halts on every input, and $O(\langle M_1\rangle, \langle M_2\rangle)$ accepts if and only if $\varepsilon \in L(M_1) \cup L(M_2)$. The trick is that nothing stops us from handing the oracle the same machine twice: $L(M') \cup L(M') = L(M')$, so the oracle then just tells us whether M' accepts ε.

Input: $(\langle M\rangle, x)$, where M is a Turing machine and x a string
Output: accept if $M(x)$ accepts, reject otherwise

1: **function** $M_{\text{ACC}}(\langle M\rangle, x)$
2: Construct the code $\langle M'\rangle$ of the following machine M':
function $M'(w)$: ▷ w is ignored
Run $M(x)$ ▷ $\langle M\rangle$ and x are hard-coded into M'
if $M(x)$ accepts **then return** accept **else return** reject
3: **return** $O(\langle M'\rangle, \langle M'\rangle)$
4: **end function**

M_{ACC} always halts. Writing down $\langle M'\rangle$ is string manipulation (a fixed template with $\langle M\rangle$ and x pasted in), and M_{ACC} never runs M or M'. It then makes one call to O, which halts because O decides L, and returns that answer.

Key property of M'. As in part (a), $M'(w)$ ignores w and does what $M(x)$ does: it accepts if $M(x)$ accepts, rejects if $M(x)$ rejects, and loops if $M(x)$ loops. So $\varepsilon \in L(M')$ if and only if $M(x)$ accepts, and since $L(M') \cup L(M') = L(M')$,

$$\varepsilon \in L(M') \cup L(M') \iff M(x) \text{ accepts.}$$

Correctness. By the definition of L_{ACC}, the key property, the definition of L, the fact that O decides L, and the code of M_{ACC},

$$\begin{aligned}
(\langle M\rangle, x) \in L_{\text{ACC}} &\implies M(x) \text{ accepts} \\
&\implies \varepsilon \in L(M') = L(M') \cup L(M') \\
&\implies (\langle M'\rangle, \langle M'\rangle) \in L \\
&\implies O(\langle M'\rangle, \langle M'\rangle) \text{ accepts} \\
&\implies M_{\text{ACC}} \text{ accepts,}
\end{aligned}$$

$$\begin{aligned}
(\langle M\rangle, x) \notin L_{\text{ACC}} &\implies M(x) \text{ rejects or loops} \\
&\implies M'(\varepsilon) \text{ does not accept, so } \varepsilon \notin L(M') \cup L(M') \\
&\implies (\langle M'\rangle, \langle M'\rangle) \notin L \\
&\implies O(\langle M'\rangle, \langle M'\rangle) \text{ rejects} \\
&\implies M_{\text{ACC}} \text{ rejects.}
\end{aligned}$$

So M_{ACC} decides L_{ACC} with an oracle for L, i.e., $L_{\text{ACC}} \leq_T L$. A decider for L would therefore give a decider for L_{ACC}, which does not exist, so L is undecidable.

(One could also reduce from part (a): $L_{\varepsilon\text{-ACC}} \leq_T L$ via the one-line machine that returns $O(\langle M\rangle, \langle M\rangle)$. I reduced from L_{ACC} directly so that the starting point is a language proved undecidable in lecture.)

Q4.1 HALTs (a)

$L_{\text{BOUNDED-HALT}}$ is **decidable**.

The point is that both parts of the condition are bounded by numbers we can read off the input. Let Σ be the input alphabet of M (it is listed in $\langle M \rangle$). There are only finitely many strings over Σ of length at most k, namely $\sum_{\ell=0}^{k} |\Sigma|^{\ell}$ of them, and for each one we only need to watch M for 376 steps. So we can just try them all.

Input: $(\langle M \rangle, k)$, where M is a Turing machine and $k \geq 0$ an integer
Output: accept if some input x with $|x| \leq k$ has $M(x)$ halting within 376 steps, reject otherwise

```
function D(⟨M⟩, k)
    for ℓ = 0, 1, ..., k do
        for each string x ∈ Σ^ℓ (in alphabetical order) do
            Simulate M on x for at most 376 steps
            if M entered its accept or reject state during these steps then return accept
            end if
        end for
    end for
    return reject
end function
```

D always halts. The two loops go over finitely many strings: $|\Sigma|^{\ell}$ strings for each $\ell \leq k$, and k is a fixed number given in the input. Each simulation is cut off after at most 376 steps of M, and simulating one step of M (look up the transition for the current state and symbol, update the tape and head) takes finite time. So every iteration takes finitely many steps, there are finitely many iterations, and D returns. D never runs M without a step bound.

Correctness. The simulation is exact, so for each x, M halts within the first 376 simulated steps if and only if $M(x)$ really halts in at most 376 steps.

- If $(\langle M \rangle, k) \in L_{\text{BOUNDED-HALT}}$, there is some x^* with $|x^*| \leq k$ such that $M(x^*)$ halts in at most 376 steps. The loops eventually reach x^* (unless D has already accepted on an earlier string, which is also fine), the simulation of $M(x^*)$ sees M halt, and D accepts.
- If $(\langle M \rangle, k) \notin L_{\text{BOUNDED-HALT}}$, then for every x with $|x| \leq k$, $M(x)$ does not halt within 376 steps. So no simulation sees a halting state, the **if** never fires, the loops run to completion, and D rejects.

So D decides $L_{\text{BOUNDED-HALT}}$, and $L_{\text{BOUNDED-HALT}}$ is decidable.

(In 376 steps M can only read the first 376 cells of its tape, so it would be enough to try strings of length at most $\min(k, 376)$, but the search above is already finite, so I did not bother.)

Q4.2 HALTs (b)

$L_{\text{LINEAR-HALT}}$ is **undecidable**. I show $L_{\text{HALT}} \leq_T L_{\text{LINEAR-HALT}}$; since L_{HALT} is undecidable (lecture), so is $L_{\text{LINEAR-HALT}}$.

Unlike part (a), the step budget $|w|$ is not fixed: it grows with the input. So a machine that takes the same number T of steps on every input halts "in time" on every input of length at least T.

Let O be an oracle that decides $L_{\text{LINEAR-HALT}}$: O halts on every input, and $O(\langle N \rangle)$ accepts if and only if there is some string w such that $N(w)$ halts in at most $|w|$ steps.

Input: $(\langle M \rangle, x)$, where M is a Turing machine and x a string
Output: accept if $M(x)$ halts, reject otherwise

1: **function** $M_{\text{HALT}}(\langle M \rangle, x)$
2: Construct the code $\langle M' \rangle$ of the following machine M':
function $M'(w)$: ▷ w is never read
Run $M(x)$ ▷ $\langle M \rangle$ and x are hard-coded into M'
return accept
3: **return** $O(\langle M' \rangle)$
4: **end function**

M_{HALT} always halts. Writing down $\langle M' \rangle$ is string manipulation, and M_{HALT} never runs M or M'. It then makes one call to O, which halts because O decides $L_{\text{LINEAR-HALT}}$, and returns that answer.

Key property of M'. M' never looks at its input, so its computation is literally the same on every input w. (It must not spend $|w|$ steps erasing w first, and it does not need to: M' writes x over the first $|x|$ cells using a marked copy of M's tape alphabet, and during the simulation treats every unmarked cell, leftover symbol of w or blank, as a blank.) So if $M(x)$ halts, there is a number $T \geq 1$, not depending on w, such that $M'(w)$ halts after exactly T steps for every w; and if $M(x)$ loops, then $M'(w)$ loops for every w.

Correctness. Let a be any symbol of the input alphabet. By the definition of L_{HALT}, the key property, the definition of $L_{\text{LINEAR-HALT}}$, the fact that O decides $L_{\text{LINEAR-HALT}}$, and the code of M_{HALT},

$$
\begin{aligned}
(\langle M \rangle, x) \in L_{\text{HALT}} &\implies M(x) \text{ halts} \\
&\implies M' \text{ halts on every } w \text{ in exactly } T \text{ steps, so on } w = a^T \text{ in } T = |w| \text{ steps} \\
&\implies \langle M' \rangle \in L_{\text{LINEAR-HALT}} \\
&\implies O(\langle M' \rangle) \text{ accepts} \\
&\implies M_{\text{HALT}} \text{ accepts,}
\end{aligned}
$$

$$
\begin{aligned}
(\langle M \rangle, x) \notin L_{\text{HALT}} &\implies M(x) \text{ loops} \\
&\implies M' \text{ loops on every } w, \text{ so no } w \text{ has } M' \text{ halting within } |w| \text{ steps} \\
&\implies \langle M' \rangle \notin L_{\text{LINEAR-HALT}} \\
&\implies O(\langle M' \rangle) \text{ rejects} \\
&\implies M_{\text{HALT}} \text{ rejects.}
\end{aligned}
$$

So M_{HALT} decides L_{HALT} using an oracle for $L_{\text{LINEAR-HALT}}$, i.e., $L_{\text{HALT}} \leq_T L_{\text{LINEAR-HALT}}$. A decider for $L_{\text{LINEAR-HALT}}$ would give a decider for L_{HALT}, which does not exist, so $L_{\text{LINEAR-HALT}}$ is undecidable.

Q5.1 Recognizing Unrecognizability (a)

L_{LOOPS} is **unrecognizable**.

The reason is that L_{LOOPS} is essentially the complement of L_{HALT}, and L_{HALT} is recognizable but not decidable. We saw in lecture that if both a language and its complement are recognizable, then the language is decidable. I include the argument below, since it is the whole proof.

Setup. Let V be the set of strings that are valid encodings of a pair $(\langle M\rangle, x)$ with M a Turing machine and x a string. V is decidable by a syntactic check, as in Q2(c). For every pair in V, $M(x)$ either halts or loops, and not both, so

$$L_{\text{HALT}} \cup L_{\text{LOOPS}} = V \qquad \text{and} \qquad L_{\text{HALT}} \cap L_{\text{LOOPS}} = \emptyset.$$

Also, L_{HALT} is recognizable (lecture): the universal machine $U(\langle M\rangle, x)$ runs M on x and accepts as soon as it halts, so U accepts exactly the pairs in L_{HALT}.

Suppose for contradiction that L_{LOOPS} is recognizable, and let R be a machine that accepts exactly the strings in L_{LOOPS}. Consider the following machine.

Input: A string w
Output: accept if $w \in L_{\text{HALT}}$, reject otherwise

1: **function** $D(w)$
2: **if** $w \notin V$ **then return** reject ▷ not a valid pair; decidable check
3: **end if**
4: **loop**
5: Run $U(w)$ for one more step; **if** U has accepted **then return** accept
6: Run $R(w)$ for one more step; **if** R has accepted **then return** reject
7: **end loop**
8: **end function**

(Both simulations keep their own tape and state between iterations, so this alternates steps of $U(w)$ and $R(w)$; if one of them halts without accepting, D just keeps stepping the other one.)

D always halts. If $w \notin V$, D rejects right away. If $w \in V$, then $w \in L_{\text{HALT}}$ or $w \in L_{\text{LOOPS}}$. In the first case $U(w)$ accepts after some finite number t of steps, so D returns by the end of iteration t. In the second case $R(w)$ accepts after some finite number of steps, so again D returns. Either way the loop stops.

Correctness.

- If $w \in L_{\text{HALT}}$, then $w \in V$, $U(w)$ accepts, and $R(w)$ never accepts (because $w \notin L_{\text{LOOPS}}$ and R recognizes L_{LOOPS}). So the only way out of the loop is "return accept," and D accepts.
- If $w \notin L_{\text{HALT}}$, then either $w \notin V$ and D rejects on line 1, or $w \in V$ with $M(x)$ looping, i.e., $w \in L_{\text{LOOPS}}$. Then $R(w)$ accepts and $U(w)$ never accepts, so D rejects.

So D decides L_{HALT}. But L_{HALT} is undecidable (lecture). This contradiction shows that no such R exists: L_{LOOPS} is unrecognizable.

Q5.2 Recognizing Unrecognizability (b)

L_{S+} is **recognizable** (for every fixed finite set S).

Write $S = \{s_1, s_2, \ldots, s_n\}$; since S is fixed and finite, this list can be hard-coded into the recognizer. Note $\langle M \rangle \in L_{S+}$ if and only if M accepts every one of $s_1, \ldots, s_n$. The recognizer just runs M on each s_i in turn. It may get stuck forever on some s_i, but that only happens when M does not accept s_i, so it is allowed.

Input: A string w
Output: accept if $w = \langle M \rangle$ with $L(M) \supseteq S$; otherwise reject or loop

1: **function** $R(w)$
2: **if** w is not a valid encoding $\langle M \rangle$ of a Turing machine **then return** reject
3: **end if**
4: **for** $i = 1, 2, \ldots, n$ **do**
5: Run $M(s_i)$ ▷ this may loop, which is allowed for a recognizer
6: **if** $M(s_i)$ rejects **then return** reject
7: **end if**
8: **end for**
9: **return** accept ▷ every s_i was accepted
10: **end function**

A recognizer does not have to halt on every input, so what has to be shown is that R accepts w if and only if $w \in L_{S+}$.

If $w \in L_{S+}$, then R accepts w. Then $w = \langle M \rangle$ for a Turing machine M with $L(M) \supseteq S$, so the validity check passes, and M accepts every s_i. So in iteration i the simulation of $M(s_i)$ halts, in the accept state, and the **if** does not fire. After all n iterations, R reaches the last line and accepts. (If $S = \emptyset$, the loop is empty and R accepts every valid $\langle M \rangle$, which is right, since every $L(M) \supseteq \emptyset$.)

If $w \notin L_{S+}$, then R does not accept w. If w is not a valid encoding, R rejects on line 1. Otherwise $w = \langle M \rangle$ and $L(M) \not\supseteq S$, so there is some $s_j \in S$ that M does not accept. Take the smallest such j. For $i < j$, $M(s_i)$ accepts, so iterations $1, \ldots, j-1$ finish without returning. In iteration j, $M(s_j)$ either rejects, in which case R rejects, or loops, in which case R loops inside the simulation and never reaches the accept line. Either way R does not accept.

So R accepts exactly the strings in L_{S+}, i.e., R recognizes L_{S+}.

(The order in which the s_i are tried does not matter, and running the n simulations in parallel would also work. Running them one after another is simpler and is enough, because the only inputs where R gets stuck are inputs it is supposed to not accept anyway.)

Q6 Lagging Barber

Recall $L_{\text{BARBER}} = \{\langle M\rangle : M(\langle M\rangle) \text{ does not accept}\}$, which is undecidable (lecture). I show $L_{\text{BARBER}} \leq_T L_{\text{LAG}}$, so L_{LAG} is undecidable as well.

Let O be an oracle that decides L_{LAG}: O halts on every input, and $O(\langle M_1\rangle, \langle M_2\rangle)$ accepts if and only if for every $x \in \Sigma^*$, $M_2(x)$ accepts implies $M_1(x)$ loops.

Idea. Choose M_1 to be a machine that *never* loops. Then "$M_2(x)$ accepts $\Rightarrow$ $M_1(x)$ loops" can only hold when $M_2(x)$ does not accept, so M_1 lags behind M_2 exactly when M_2 accepts nothing. Now choose M_2 to accept everything if $M(\langle M\rangle)$ accepts, and nothing otherwise; then the oracle answers exactly the Barber question.

Input: $\langle M\rangle$, where M is a Turing machine
Output: accept if $M(\langle M\rangle)$ does not accept, reject otherwise

1: **function** $M_{\text{BARBER}}(\langle M\rangle)$
2: Let M_{yes} be the machine that accepts every input immediately (fixed code)
3: Construct the code $\langle M'\rangle$ of the following machine M':
function $M'(w)$: ▷ w is ignored
Run $M(\langle M\rangle)$ ▷ $\langle M\rangle$ is hard-coded into M'
if $M(\langle M\rangle)$ accepts **then return** accept **else return** reject
4: **return** $O(\langle M_{\text{yes}}\rangle, \langle M'\rangle)$ ▷ $M_1 = M_{\text{yes}}$, $M_2 = M'$
5: **end function**

M_{BARBER} always halts. $\langle M_{\text{yes}}\rangle$ is a fixed string, and writing down $\langle M'\rangle$ is string manipulation with $\langle M\rangle$ pasted into a template. M_{BARBER} never runs M, M', or M_{yes}. It makes one call to O, which halts because O decides L_{LAG}, and returns that answer.

Key properties. $M_{\text{yes}}(x)$ halts on every x, so M_{yes} never loops. $M'(w)$ ignores w and does what $M(\langle M\rangle)$ does, so either M' accepts every string (if $M(\langle M\rangle)$ accepts) or M' accepts no string (if $M(\langle M\rangle)$ rejects or loops).

Correctness. By the definition of L_{BARBER}, the key properties, the definition of lagging, the fact that O decides L_{LAG}, and the code of M_{BARBER},

$$\begin{aligned}
\langle M\rangle \in L_{\text{BARBER}} &\Longrightarrow M(\langle M\rangle) \text{ does not accept} \\
&\Longrightarrow M'(x) \text{ does not accept, for every } x \in \Sigma^* \\
&\Longrightarrow \text{“}M'(x) \text{ accepts} \Rightarrow M_{\text{yes}}(x) \text{ loops” holds vacuously for every } x \\
&\Longrightarrow M_{\text{yes}} \text{ lags behind } M', \text{ i.e., } (\langle M_{\text{yes}}\rangle, \langle M'\rangle) \in L_{\text{LAG}} \\
&\Longrightarrow O(\langle M_{\text{yes}}\rangle, \langle M'\rangle) \text{ accepts} \\
&\Longrightarrow M_{\text{BARBER}} \text{ accepts,}
\end{aligned}$$

$$\begin{aligned}
\langle M\rangle \notin L_{\text{BARBER}} &\Longrightarrow M(\langle M\rangle) \text{ accepts} \\
&\Longrightarrow M'(\varepsilon) \text{ accepts, but } M_{\text{yes}}(\varepsilon) \text{ halts rather than loops} \\
&\Longrightarrow \text{the lag condition fails at } x = \varepsilon, \text{ i.e., } (\langle M_{\text{yes}}\rangle, \langle M'\rangle) \notin L_{\text{LAG}} \\
&\Longrightarrow O(\langle M_{\text{yes}}\rangle, \langle M'\rangle) \text{ rejects} \\
&\Longrightarrow M_{\text{BARBER}} \text{ rejects.}
\end{aligned}$$

So M_{BARBER} decides L_{BARBER} using an oracle for L_{LAG}, i.e., $L_{\text{BARBER}} \leq_T L_{\text{LAG}}$. If L_{LAG} were decidable, replacing O by its decider would give a decider for L_{BARBER}, which does not exist. Therefore L_{LAG} is undecidable.

Q7.1 Infinite machines, infinite inputs! (a)

The flaw is that the test for M_1 never finishes if M_1 does not halt on any input, so the procedure never gets to M_2.

Dovetailing over the inputs $s_1, s_2, s_3, \ldots$ of M_1 does fix one problem: no single looping computation can block the others, since every $M_1(s_j)$ keeps getting steps. And if M_1 halts on some s_j, the dovetailing will eventually see that, because after enough rounds $M_1(s_j)$ has been given as many steps as it needs. But there are infinitely many inputs. If M_1 loops on every one of them, then no round ever produces a halt, and the dovetailing just keeps starting new computations $M_1(s_4), M_1(s_5), \ldots$ forever. There is no point at which Professor Υ can declare "M_1 is done, move on to M_2," because "M_1 halts on no input" is not something that can be observed in finitely many steps.

So take a collection in which M_1 loops on everything (say M_1 immediately enters an infinite loop), while M_2 halts on s_1 in one step. The correct answer is "yes, some machine halts on some input," but Professor Υ's procedure spends forever on M_1 and never runs a single step of M_2. Dovetailing has to cover *both* infinite dimensions, machines and inputs, at the same time; dovetailing over one of them and then iterating over the other does not work.

(There is a smaller issue in the phrasing as well: Professor Υ talks about testing whether M_1 "halts on all its possible inputs," but the goal only needs some machine to halt on some input, which is what the dovetailing can detect. Halting on *all* inputs is not something any finite amount of simulation can confirm either.)

Q7.2 Infinite machines, infinite inputs! (b)

The flaw is the same one, with the roles of machines and inputs swapped: the test for s_1 never finishes if no machine halts on s_1, so the procedure never gets to s_2.

Dovetailing over the machines $M_1, M_2, M_3, \ldots$ on the fixed input s_1 makes sure that every $M_i(s_1)$ keeps getting steps, so if some M_i halts on s_1, that will eventually be seen. But there are infinitely many machines. If every M_i loops on s_1, then no round ever produces a halt, and Dr. Epsiwrong's procedure keeps adding $M_4(s_1), M_5(s_1), \ldots$ to the pile forever. The event "no machine halts on s_1" cannot be observed after finitely many steps, so there is never a moment to say "done with s_1, move on to s_2."

Concretely, take a collection in which every M_i loops on s_1 (say each M_i loops whenever its input is s_1), while M_1 halts on s_2 in one step. The correct answer is "yes," but the procedure spends forever on s_1 and never runs a single step on s_2.

So both colleagues made the same mistake. Each of them dovetails along one infinite list and then tries to iterate over the other infinite list, and the iteration never advances past its first element. The fix, in part (c), is to dovetail over machines and inputs simultaneously, so that every pair (M_i, s_j) gets started after finitely many rounds and then gets a step in every round after that.

Q7.3 Infinite machines, infinite inputs! (c)

Setup. Let $s_1, s_2, s_3, \ldots$ be the standard enumeration of Σ^* (by length, and alphabetically within each length); given j, the string s_j can be computed. I assume the archive gives us access to the machines in order, i.e., given i we can look up the code $\langle M_i \rangle$ (for example, the encodings are stored one after another on a read-only tape). Without some such access there is nothing to simulate. The machine below ignores its own input.

Input: Any string (ignored)
Output: accept if some M_i halts on some s_j; loop otherwise

1: **function** A
2: **for** $n = 1, 2, 3, \ldots$ **do** ▷ round n
3: **for** each pair (i, j) with $1 \le i \le n$ and $1 \le j \le n$ **do**
4: **if** the pair (i, j) has not been started yet **then** look up $\langle M_i \rangle$ and s_j and set up the initial configuration of M_i on s_j
5: **end if**
6: Simulate one more step of $M_i(s_j)$, continuing from its saved configuration
7: **if** $M_i(s_j)$ has halted (accept or reject) **then return** accept
8: **end if**
9: **end for**
10: **end for**
11: **end function**

At any time the machine stores the configurations of the finitely many pairs started so far (at most n^2 of them during round n), so each round is a finite amount of work.

When a pair gets its steps. The pair (i, j) first appears in round $\max(i, j)$ and gets exactly one step in that round and in every later round. So by the end of round $\max(i, j) + t - 1$, the simulation of $M_i(s_j)$ has been run for exactly t steps (unless A accepted earlier).

Each round finishes. Round n handles n^2 pairs, and for each one it does one lookup (only the first time), one simulated step, and one check. All of these take finite time, so round n ends after finitely many steps and round $n + 1$ begins, unless A returns during round n.

Correctness.

- Suppose some machine in the list halts on some input: $M_i(s_j)$ halts, say after t steps. Since every round finishes, A reaches round $\max(i, j) + t - 1$ unless it has accepted before that. By then $M_i(s_j)$ has been simulated for t steps, so the simulation has reached a halting state, and the check after that step makes A accept. So A accepts.

- Suppose no machine in the list halts on any input. Then no simulated computation $M_i(s_j)$ ever reaches a halting state, so the check never fires and "return accept" is never executed. Every round finishes and the next one starts, so A runs through rounds $1, 2, 3, \ldots$ forever. So A loops.

Hence A accepts if and only if some M_i halts on some s_j, and loops otherwise, as required.

(Note that A is a recognizer, not a decider: it never answers "no." That is all the problem asks for.)

Q8.1 (Optional extra credit) (a)

True. For every undecidable L_1 and finite L_2, $L_1 \cup L_2$ is undecidable.

The idea: adding finitely many strings cannot make an undecidable language decidable, because a decider for the union, together with a finite table of answers for those finitely many strings, would decide L_1. I write this as $L_1 \leq_T L_1 \cup L_2$.

Let $L_2 = \{y_1, \ldots, y_m\}$, and let $T = L_1 \cap L_2$. T is a subset of a finite set, so T is finite, and a finite language is decided by a machine that hard-codes its members. We do not need to know which y_i are in T; it is enough that the finite list exists, so the machine D_T deciding T exists (this is the same kind of argument as for $L_{\text{SmallHalt}}$ in HW6). Let O be an oracle that decides $L_1 \cup L_2$.

Input: A string x
Output: accept if $x \in L_1$, reject otherwise

1: **function** $M_1(x)$
2: **if** $x \in \{y_1, \ldots, y_m\}$ **then return** $D_T(x)$ ▷ finite table lookup
3: **else return** $O(x)$
4: **end if**
5: **end function**

M_1 **always halts.** Comparing x against the m fixed strings is finite work, D_T halts on every input because it is a decider for a finite language, and O halts on every input because it decides $L_1 \cup L_2$.

Correctness.

- If $x \in L_2$: then $x \in L_1 \iff x \in L_1 \cap L_2 = T$, and D_T decides T, so $M_1(x)$ accepts if $x \in L_1$ and rejects otherwise.
- If $x \notin L_2$: then $x \in L_1 \iff x \in L_1 \cup L_2$ (a string outside L_2 is in the union exactly when it is in L_1), and O decides $L_1 \cup L_2$, so again $M_1(x)$ accepts if $x \in L_1$ and rejects otherwise.

So M_1 decides L_1 with an oracle for $L_1 \cup L_2$, i.e., $L_1 \leq_T L_1 \cup L_2$. If $L_1 \cup L_2$ were decidable, replacing O by its decider would give a decider for L_1, contradicting the assumption that L_1 is undecidable. Hence $L_1 \cup L_2$ is undecidable.

Q8.2 (Optional extra credit) (b)

False. An undecidable language can be made decidable by taking a union with a decidable one. Counterexample: $L_1 = L_{\text{HALT}}$ and $L_2 = \Sigma^*$.

- $L_1 = L_{\text{HALT}}$ is undecidable (lecture).
- $L_2 = \Sigma^*$ is decidable: the machine that accepts every input immediately halts on every input and accepts exactly the strings in Σ^*.
- $L_1 \cup L_2 = L_{\text{HALT}} \cup \Sigma^* = \Sigma^*$, which is decidable by the same machine.

So here L_1 is undecidable, L_2 is decidable, and $L_1 \cup L_2$ is decidable. This disproves the statement.

This does not contradict part (a). The argument there used that L_2 is finite, so that the strings where the union does not determine L_1 could be handled by a finite table. When L_2 is infinite, the union may hide infinitely much information about L_1, as it does here: Σ^* swallows L_{HALT} completely.